SOFTWARE TEST PLAN (STP)

Event Management and Registration System (EMRS)

Field	Value
Project	Event Management and Registration System (EMRS)
Version	1.1
Date	16-10-2026
Status	Draft / Submission Version
Project Manager	Aariz
Development Lead	Dhanraj Rathod
Product Owner	Vaishali Krishnan
QA / Tester	Avinash Julakatti

1. Introduction

1.1 Purpose

This document defines the test plan for the Event Management and Registration System (EMRS) v1.0. It describes the testing objectives, scope, strategy, test items, environment, responsibilities, risks, traceability, and reporting approach used to verify the requirements specified in the EMRS SRS.

1.2 Scope

Testing covers organizer account and event management, attendee event discovery and registration, waitlist management, simulated payment processing, email notifications, QR e-ticket generation and check-in, organizer analytics/reporting, platform administration/moderation, performance, availability, usability/accessibility, maintainability, and the security requirements defined in the SRS.

Out of scope are physical venue infrastructure and the internal implementation of third-party payment gateways beyond their published API, as stated in the SRS.

1.3 References

- EMRS Software Requirements Specification (SRS), Version 1.0, dated 03-09-2026.
- EMRS Software Architecture and Design Specification / component design (team project document).
- Requirements Traceability Matrix (RTM) in the EMRS SRS.
- EMRS project repository: https://github.com/Event-Management-Team7/Event_Management_Team7 (source control and CI/CD host).

1.4 Definitions

- EMRS – Event Management and Registration System
- SRS – Software Requirements Specification
- RTM – Requirements Traceability Matrix
- QR – Quick Response
- JWT – JSON Web Token
- RBAC – Role-Based Access Control
- TLS – Transport Layer Security
- NFR – Non-Functional Requirement
- CI – Continuous Integration
- CD – Continuous Delivery/Deployment
- PR – Pull Request

- SAST – Static Application Security Testing

2. Test Items

- User registration, login/authentication, password reset, and profile management
- Event creation, update, cancellation, browsing/search, and event details
- Event registration (RSVP), cancellation, and waitlist management
- Simulated payment processing
- Email notifications and reminders
- e-Ticket / QR-code generation and event check-in
- Organizer dashboard and reporting
- Admin user and event moderation
- Post-event feedback
- Security controls: password hashing, TLS, RBAC, rate limiting/CAPTCHA, and payment-data handling
- Performance, availability, scalability, usability/accessibility, and maintainability requirements

3. Features to be Tested

Req ID	Feature	Priority	Planned Test Case(s)
EMRS-F-001	User Registration	High	TC-Auth-01, TC-Auth-02
EMRS-F-002	User Login / Authentication	High	TC-Auth-03, TC-Auth-04
EMRS-F-003	Password Reset	Medium	TC-Auth-05
EMRS-F-004	User Profile Management	Medium	TC-Profile-01
EMRS-F-005	Create Event	High	TC-Event-01, TC-Event-02
EMRS-F-006	Edit / Update Event	High	TC-Event-03
EMRS-F-007	Cancel Event	High	TC-Event-04
EMRS-F-008	Browse / Search Events	High	TC-Event-05
EMRS-F-009	View Event Details	Medium	TC-Event-06
EMRS-F-010	Register for Event (RSVP)	High	TC-Reg-01, TC-Reg-02
EMRS-F-011	Cancel Registration	Medium	TC-Reg-03
EMRS-F-012	Waitlist Management	Medium	TC-Reg-04
EMRS-F-013	Payment Processing (simulated)	High	TC-Pay-01, TC-Pay-02
EMRS-F-014	Notifications & Reminders	High	TC-Notif-01, TC-Notif-02
EMRS-F-015	e-Ticket / QR Code Generation	Medium	TC-QR-01
EMRS-F-016	Event Check-in	High	TC-QR-02
EMRS-F-017	Organizer Dashboard & Reports	Medium	TC-Report-01
EMRS-F-018	Admin User & Event Moderation	Medium	TC-Admin-01, TC-Admin-02
EMRS-F-019	Post-Event Feedback	Low	TC-Feedback-01

3.1 Non-Functional and Security Features to be Tested

ID	Category	Acceptance / Verification	Test Case
EMRS-NF-001	Performance	95% of page requests complete within 2 seconds under normal load.	TC-Perf-01
EMRS-NF-002	Availability	System remains available during scheduled demonstration/testing periods, excluding planned maintenance.	TC-Avail-01
EMRS-NF-003	Scalability	Increased users/registrations during peak periods do not cause significant performance degradation.	TC-Scale-01
EMRS-NF-004	Usability & Accessibility	Responsive and usable on mobile, tablet, and desktop devices.	TC-UX-01
EMRS-NF-005	Maintainability	Modular code structure and automated testing for core functions.	TC-Maint-01
EMRS-SEC-001	Password Security	Passwords use salted bcrypt/Argon2 hashes; plaintext is not stored/logged.	TC-Sec-01
EMRS-SEC-002	Transport Security	Client-server communication uses TLS 1.2 or higher.	TC-Sec-02
EMRS-SEC-003	RBAC	Attendee, Organizer, and Admin roles are enforced on protected endpoints.	TC-Sec-03

EMRS-SEC-004	Rate Limiting / CAPTCHA	Login and registration endpoints mitigate brute-force/bot abuse.	TC-Sec-04
EMRS-SEC-005	Payment Data Protection	Card data is not stored on application servers; tokenized gateway handling is used.	TC-Sec-05

4. Features Not to be Tested

- Physical venue infrastructure and on-site facilities.
- Internal implementation of third-party payment gateways beyond the published API.
- Internal operation of third-party email service providers beyond the EMRS integration/API boundary.
- Optional Maps/Geocoding provider internals beyond the EMRS integration boundary.

5. Test Approach / Strategy

5.1 Test Levels

- Unit Testing – verify individual modules/functions such as authentication, event management, registration, and QR handling.
- Integration Testing – verify interactions between front-end, back-end API, database, notification service, and external service APIs.
- System Testing – verify end-to-end attendee, organizer, and admin workflows.
- Acceptance Testing – verify high-priority SRS requirements and acceptance criteria before submission/demo.

5.2 Test Types

- Functional testing
- Integration testing
- Regression testing (automated in the CI pipeline on every pull request and merge)
- Performance testing
- Availability/scalability testing
- Usability and accessibility testing
- Security testing
- Smoke testing (automated after each deployment to the test environment)
- Static analysis and dependency/secret scanning (automated in CI)

5.3 Entry Criteria

- A testable build is available.
- Required test data/accounts are available.
- Test environment and required services are accessible.
- Relevant functional requirements are implemented sufficiently for testing.
- Code is committed to the GitHub repository and the CI build for the commit under test completes successfully.

5.4 Exit Criteria

- 100% of planned test cases are executed.
- All high-priority functional requirements are verified.
- No critical security or NFR failures remain open.
- RTM is updated with test results.
- Acceptance criteria for the planned scope are satisfied.

- The CI/CD pipeline is green on the main branch for the release-candidate commit, and all pipeline-generated results are attached to the RTM/Test Summary Report.

5.5 Security Validation

- Verify password hashing and confirm plaintext passwords are not stored or logged.
- Verify HTTPS/TLS 1.2 or higher for client-server communication.
- Verify RBAC for Attendee, Organizer, and Admin protected operations.
- Verify rate limiting and CAPTCHA on login/registration endpoints.
- Verify that payment card data is not stored on application servers.

6. Test Environment

Area	Test Environment
Client	Modern web browser on desktop, tablet, and mobile devices as applicable.
Backend	EMRS back-end API hosted on the project's test/server environment.
Database	Relational database used by EMRS.
External Services	Email service/API; optional Maps/Geocoding API; simulated payment flow.
Network	Internet-connected environment using HTTPS/TLS.
Test Data	Dummy attendee, organizer, and admin accounts; sample events; registration/payment test data.
Source Control / CI-CD	GitHub repository (Event-Management-Team7/Event_Management_Team7) with GitHub Actions runners; protected main branch; test environment deployed automatically from main (see Section 14).
Tools	Proposed, to be confirmed once the stack is finalized: GitHub Actions (CI/CD); Jest and Supertest (unit/API); Playwright (end-to-end); k6 (load); axe-core/Lighthouse CI (accessibility); ESLint, CodeQL, npm audit and secret scanning (static/security). This assumes a Node.js/JavaScript stack, as suggested by the repository's .gitignore; equivalent tools will be used if a different stack is chosen.

7. Test Schedule

Milestone	Planned Date	Responsible
Test Plan preparation	02-Oct-2026	QA / Project Team
Test case design and RTM update	02-Oct-2026	QA / Development Team
Test environment and data setup	02-Oct-2026	Development / QA
CI/CD pipeline setup (build, test, scan, deploy-to-test)	To be confirmed	Development Lead / QA
Functional and integration execution	02-Oct-2026 onward	QA / Development Team
Defect fixing and regression testing	As defects are found	Development / QA
Acceptance verification	After planned test cases pass	Product Owner / QA

8. Test Deliverables

- Software Test Plan (this document)
- Functional and non-functional test cases
- Test data
- Test execution results/logs
- CI/CD pipeline configuration (GitHub Actions workflow files) stored in the repository
- Automated pipeline run reports and artifacts (test results, coverage, security scan output)
- Defect reports
- Updated Requirements Traceability Matrix (RTM)
- Final Test Summary Report

9. Roles and Responsibilities

Role	Name	Responsibility
Project Manager	Aariz	Coordinate project status and submission readiness.
Development Lead	Dhanraj Rathod	Support implementation, defect fixing, and technical clarification. Own and maintain the CI/CD workflows and branch protection rules.
Product Owner	Vaishali Krishnan	Confirm expected product behavior and acceptance/sign-off.
QA / Tester	Avinash Julakatti	Prepare and execute tests, record results, and track defects. Author automated test suites run by the pipeline and review pipeline results.

10. Risks and Mitigation

Risk	Impact / Cause	Mitigation
Incomplete implementation	Some requirements may not be testable if development is incomplete.	Prioritize high-priority requirements and identify gaps before execution.
Dependency on external services	Email, Maps, or payment integrations may be unavailable.	Use test/sandbox interfaces or stubs where applicable and document limitations.
Test environment instability	Server/database/network problems can block execution.	Maintain a repeatable test setup and record environment issues.
Security defects	Authentication/RBAC/payment-data issues may affect acceptance.	Run security checks before final acceptance and fix critical findings.
Limited test time	Deadline may restrict full execution.	Prioritize high-priority functional, security, and NFR tests first.
CI/CD pipeline failures or flaky tests	Unstable workflows, runner limits or flaky tests can block merges or hide real defects.	Keep workflows simple and version-controlled; quarantine and fix flaky tests promptly; re-run failed jobs before raising defects.
Secrets or test data exposure in pipeline	Credentials or API keys could leak through code or logs.	Use GitHub Secrets and environment-scoped variables; keep .env files out of Git; use dummy data and sandbox keys only.

11. Assumptions & Dependencies

- The EMRS application build will be available for testing.
- Required database and backend services will be accessible.
- Dummy user/event/test transaction data will be available.
- External email/payment/map integrations, where used, will expose test/sandbox behavior.
- The test team will have access to the roles required to verify Attendee, Organizer, and Admin behavior.
- The team has write access to the GitHub repository and can use GitHub Actions and repository secrets.
- Source code and automated tests will be committed to the repository so the pipeline can build and test them.

12. Suspension & Resumption Criteria

Suspend testing if:

- The test environment is unavailable and blocks planned execution.
- A build has blocking defects that prevent meaningful execution of a substantial portion of the test cases.
- Required database/backend services are unavailable.
- The CI pipeline on the main branch is failing because of a broken build or failing smoke tests that have not been triaged.

Resume testing if:

- Blocking defects are resolved or a stable build is delivered.
- The test environment and required services are restored.
- Required test data and user roles are available.
- The pipeline on the main branch is green again.

13. Test Case Management & Traceability

The RTM maps each SRS requirement to one or more planned test cases. Status values are N = Not Executed, P = Pass, F = Fail, and A = Aborted/Blocked. For automated tests, status is taken from the CI pipeline results (see Section 14). The current SRS already contains an initial RTM for selected requirements; the remaining mappings will be completed as the Test Plan is finalized.

Req ID	Requirement	Module	Test Case(s)	Status
EMRS-F-001	User Registration	Auth	TC-Auth-01, TC-Auth-02	N
EMRS-F-002	User Login / Authentication	Auth	TC-Auth-03, TC-Auth-04	N
EMRS-F-003	Password Reset	Auth	TC-Auth-05	N
EMRS-F-004	User Profile Management	Profile	TC-Profile-01	N
EMRS-F-005	Create Event	Event	TC-Event-01, TC-Event-02	N
EMRS-F-006	Edit / Update Event	Event	TC-Event-03	N
EMRS-F-007	Cancel Event	Event	TC-Event-04	N
EMRS-F-008	Browse / Search Events	Event	TC-Event-05	N
EMRS-F-009	View Event Details	Event	TC-Event-06	N
EMRS-F-010	Register for Event	Registration	TC-Reg-01, TC-Reg-02	N
EMRS-F-011	Cancel Registration	Registration	TC-Reg-03	N
EMRS-F-012	Waitlist Management	Registration	TC-Reg-04	N
EMRS-F-013	Simulated Payment	Payment	TC-Pay-01, TC-Pay-02	N
EMRS-F-014	Notifications & Reminders	Notification	TC-Notif-01, TC-Notif-02	N
EMRS-F-015	QR e-Ticket	Ticketing	TC-QR-01	N
EMRS-F-016	Event Check-in	Check-in	TC-QR-02	N
EMRS-F-017	Organizer Dashboard/Reports	Reporting	TC-Report-01	N
EMRS-F-018	Admin Moderation	Admin	TC-Admin-01, TC-Admin-02	N
EMRS-F-019	Post-Event Feedback	Feedback	TC-Feedback-01	N
EMRS-NF-001	Performance	System	TC-Perf-01	N
EMRS-NF-002	Availability	System	TC-Avail-01	N
EMRS-NF-003	Scalability	System	TC-Scale-01	N
EMRS-NF-004	Usability/Accessibility	UI	TC-UX-01	N
EMRS-NF-005	Maintainability	Codebase	TC-Maint-01	N
EMRS-SEC-001	Password Security	Auth	TC-Sec-01	N
EMRS-SEC-002	TLS	Communication	TC-Sec-02	N
EMRS-SEC-003	RBAC	Authorization	TC-Sec-03	N
EMRS-SEC-004	Rate Limiting/CAPTCHA	Auth	TC-Sec-04	N
EMRS-SEC-005	Payment Data Protection	Payment	TC-Sec-05	N

14. CI/CD Pipeline

14.1 Purpose and Current State

The CI/CD pipeline runs the automated tests in this plan on every change, so defects are found early and regression testing (Section 5.2) and the maintainability requirement EMRS-NF-005 (automated testing for core functions) are met continuously rather than only before submission.

At the time of this revision, the project repository contains the SRS, the component diagram, a README and a Node.js-style .gitignore, but no application source code or workflow files yet. The pipeline below is therefore a planned design; stage details and tools will be confirmed when the implementation stack is finalized and the workflow files are added to the repository.

14.2 Source Control and Branching

- The repository is hosted on GitHub; CI/CD is implemented with GitHub Actions workflow files kept under .github/workflows.
- The main branch is protected: changes are made on feature branches and merged only through reviewed pull requests whose required checks pass.
- Files such as .env are excluded by .gitignore; configuration values and credentials are supplied through GitHub Secrets.

14.3 Pipeline Stages

Flow: Commit / Pull Request, then Build, Lint and static analysis, Unit tests, Integration tests and Security scans. After merge to main: Deploy to test, then Smoke and end-to-end tests, then Test report. Nightly: scheduled NFR checks.

Stage	Trigger	Activities	Proposed Tools	Gate / Pass Criteria
1. Build	Push, PR	Install dependencies; build front end and back end.	npm ci, build scripts	Build succeeds with no errors.
2. Lint & static analysis	Push, PR	Code style checks; static security analysis (SAST).	ESLint, CodeQL	No lint errors; no high/critical findings.
3. Unit tests	Push, PR	Run unit tests for Auth, Event, Registration, Payment (simulated), Notification and Ticket modules.	Jest	All tests pass; coverage meets the agreed threshold.
4. Integration / API tests	PR	Run API tests against a test database service; email and payment replaced by stubs/sandbox.	Supertest, GitHub service containers	All tests pass, including RBAC and rate-limit checks.
5. Security scans	Push, PR, weekly	Dependency vulnerability scan; secret scan; confirm no plaintext passwords or card data are stored or logged.	npm audit, Dependabot, secret scanning	No high/critical vulnerabilities; no exposed secrets.
6. Deploy to test	Merge to main	Deploy the build to the test environment and apply database migrations/seed data.	GitHub Actions deploy job	Deployment completes and health check responds.
7. Smoke & E2E tests	After deploy	Run smoke tests and end-to-end attendee, organizer and admin flows; verify HTTPS/TLS 1.2+.	Playwright	All smoke tests pass; TLS check passes.
8. Scheduled NFR checks	Nightly / before acceptance	Load and scalability tests; accessibility checks; availability health checks.	k6, axe-core/Lighthouse CI	NFR targets met (e.g., 95% of requests within 2 seconds).

14.4 Test Case Automation Coverage

Planned test cases are mapped to pipeline stages as shown below. Usability, exploratory testing and final acceptance remain manual.

Test Case(s)	Primary CI Stage	Automation
TC-Auth-01 to 05, TC-Profile-01	Unit, integration/API	Automated
TC-Event-01 to 06	Unit, integration/API, E2E (search)	Automated
TC-Reg-01 to 04	Integration/API, E2E	Automated
TC-Pay-01 to 02	Integration/API with simulated gateway	Automated
TC-Notif-01 to 02	Integration/API with email stub or sandbox	Automated; real inbox delivery checked manually
TC-QR-01 to 02	Unit, E2E	Partly automated; camera scan checked manually
TC-Report-01, TC-Admin-01 to 02, TC-Feedback-01	Integration/API, E2E	Automated
TC-Perf-01, TC-Scale-01	Scheduled load test (Stage 8)	Automated
TC-Avail-01	Scheduled health check (Stage 8)	Automated
TC-UX-01	Accessibility scan (Stage 8) plus manual device review	Partly automated

TC-Maint-01	Unit test and coverage reports from Stage 3	Automated
TC-Sec-01, 03, 04, 05	Integration/API, security scans	Automated
TC-Sec-02	Post-deploy TLS check (Stage 7)	Automated

14.5 Quality Gates

- A pull request cannot be merged unless Stages 1 to 5 pass.
- A failed deployment, smoke test or TLS check on main marks the build as not ready for acceptance testing.
- Any critical or high security finding must be fixed or formally accepted by the Product Owner before final acceptance (see Section 5.4).

14.6 Reporting and Evidence

- Each run publishes test results, coverage and scan reports as pipeline artifacts, retained for the test period.
- QA uses these results as execution evidence when updating the RTM and the final Test Summary Report. As stated in the note at the end of Section 13, a requirement is marked Pass only after its test has actually run and evidence is recorded.
- Failed runs are triaged by QA and the Development Lead; confirmed defects are logged and re-tested by re-running the pipeline.

15. Test Metrics & Reporting

- % of planned test cases executed
- % of test cases passed / failed / blocked
- Number of critical, high, medium, and low severity defects
- Defect aging
- Requirement coverage in the RTM
- Pass rate for high-priority functional requirements
- Security and NFR test pass status
- CI build success rate and average pipeline duration
- Automated test pass rate and code coverage (from pipeline reports)
- Number of failed pipeline runs on main and mean time to fix

Reports: daily execution/status update during the test period and a final Test Summary Report.

16. Approvals

Role	Name	Signature	Date
QA / Tester	Avinash Julakatti	_____	_____
Development Lead	Dhanraj Rathod	_____	_____
Product Owner	Vaishali Krishnan	_____	_____
Project Manager	Aariz	_____	_____

Note: Test results are initially marked N (Not Executed). Do not change them to Pass until the corresponding test has actually been executed and evidence/results have been recorded.